

**SIMATS ENGINEERING**  
**CO3 - ASSESSMENT TOOL - 2: Case Study**

**COURSE NAME:** Cloud Computing and Big Data Analytics      **COURSE CODE:** CSA1522

|                 |                      |
|-----------------|----------------------|
| Name            | Atmakuri Sai Mrudula |
| Register Number | 192372087            |

**COURSE OUTCOME COVERED**

**CO3:** *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)  
Question Count: 3

**Total Marks: 30**

---

**Code of Conduct**

I Atmakuri Sai Mrudula/192372087 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Sai Mrudula

**Assessment Tasks - Case Study**

**1. Case Study 1: Smart Retail Inventory Management System**

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

**2. Case Study 2: Cloud-Based Document Collaboration System**

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

**3. Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced

through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

### Case Study Rubric

| Criteria                | Weightage | Level 4 - Exemplary                                        | Level 3 - Proficient                | Level 2 - Developing     | Level 1 - Beginning           |
|-------------------------|-----------|------------------------------------------------------------|-------------------------------------|--------------------------|-------------------------------|
| Understanding of Case   | 25%       | Clearly identifies all technical and business requirements | Identifies most requirements        | Basic understanding only | Poor understanding of case    |
| Application of Concepts | 25%       | Applies suitable cloud concepts and tools effectively      | Good application with minor issues  | Partial application      | Weak or incorrect application |
| Analysis                | 20%       | Strong analysis with clear trade-off reasoning             | Reasonable analysis                 | Limited analysis         | Minimal analysis              |
| Solution Design         | 20%       | Solution is feasible, practical, and well justified        | Reasonable solution with minor gaps | Basic solution           | Unclear solution              |
| Clarity                 | 10%       | Well-structured and easy to follow                         | Mostly clear                        | Somewhat unclear         | Poorly organized              |

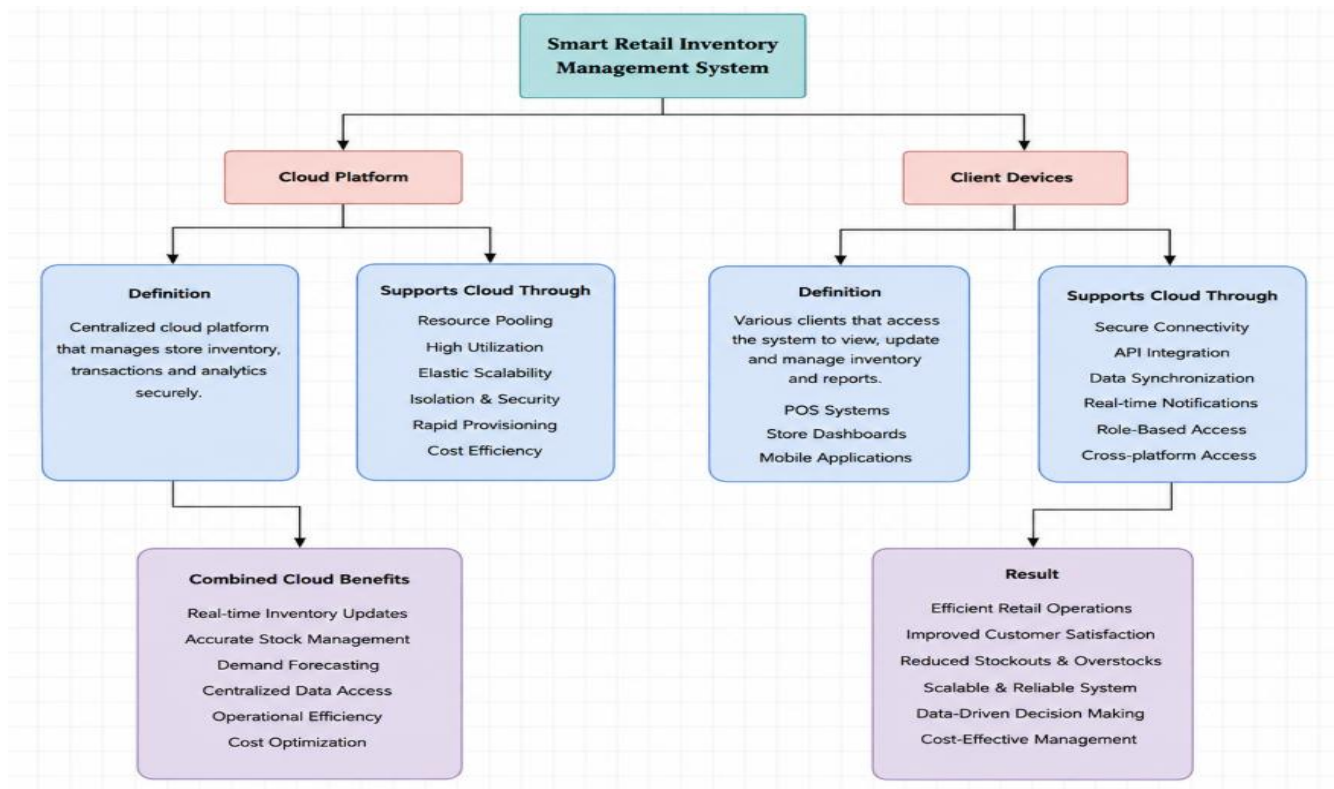
## 1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

A Smart Retail Inventory Management System is a cloud-based application that helps retail stores monitor stock levels, sales, and customer demand in real time. It connects Point of Sale (POS) systems, store dashboards, and cloud services through secure APIs. The system improves inventory accuracy, reduces manual work, and supports better business decisions using cloud analytics.

### Architecture Components: -

- **Clients:** POS systems, store dashboards, and mobile applications.
- **Web APIs:** REST APIs connect client applications with cloud services.
- **Cloud Platform Services:** Inventory management, demand forecasting, and analytics.
- **Cloud Storage:** Stores product details, sales records, transaction logs, and analytics datasets.
- **Infrastructure:** Virtual machines, load balancers, and networking services.
- **Security:** VPN, firewall, API gateway, authentication, and authorization.



**Working Process: -**

1. A customer purchases a product at the POS system.
2. The POS sends the transaction to the cloud through REST APIs.
3. The cloud updates inventory levels instantly.
4. Product and transaction information is stored in cloud databases.
5. Analytics services analyze sales patterns.
6. Demand forecasting predicts future stock requirements.
7. Store managers monitor inventory using web dashboards.
8. Security services protect communication using VPNs, firewalls, and API gateways.

**Cloud Services Used: -**

- **IaaS:** Virtual machines, networking, storage.
- **PaaS:** Analytics platform, API services, forecasting services.
- **SaaS:** Inventory dashboard and reporting applications.

**Advantages: -**

- Real-time inventory updates.
- Accurate demand forecasting.
- Reduced stock shortages and overstocking.
- Centralized data management.
- High scalability and availability.
- Secure communication between stores and cloud.

The Smart Retail Inventory Management System integrates cloud infrastructure, APIs, storage, and analytics to provide efficient inventory management. It enables retailers to improve operational efficiency, reduce costs, and enhance customer satisfaction.

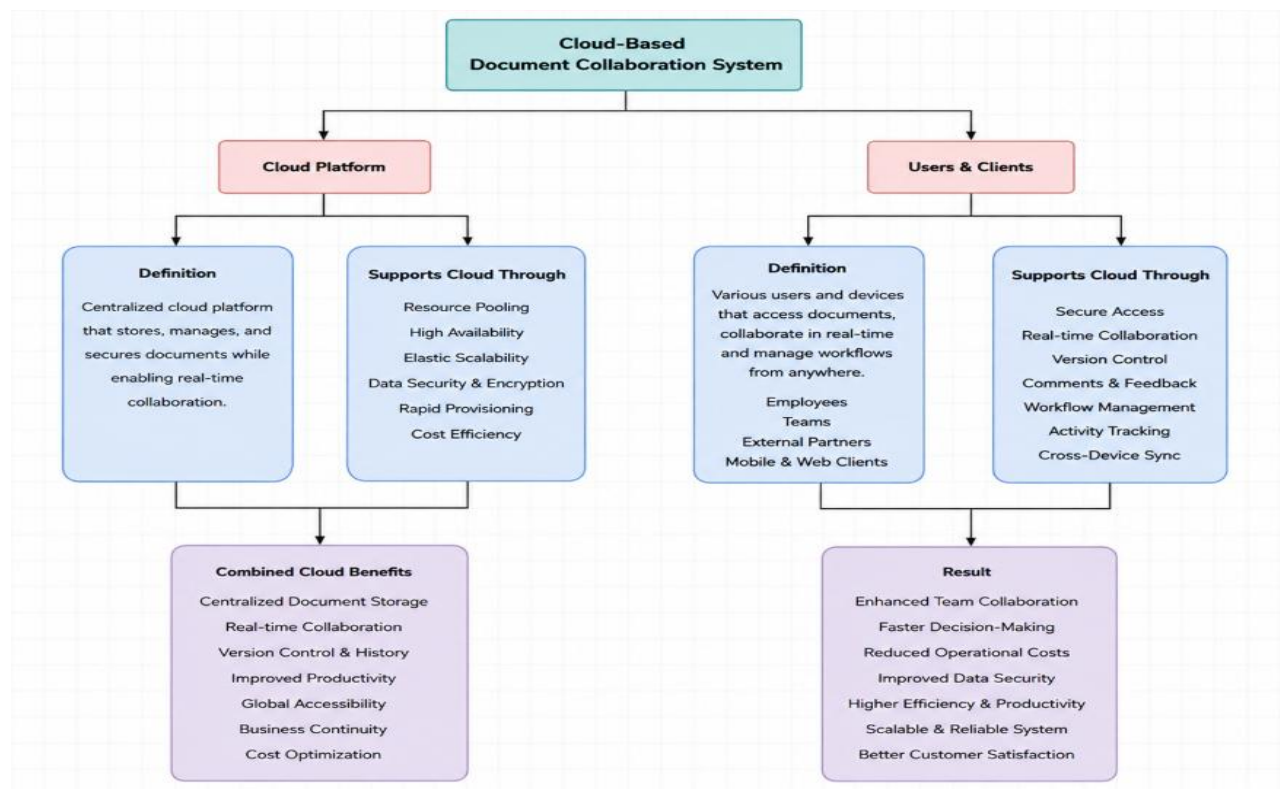
## 2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

A Cloud-Based Document Collaboration System enables multiple users to create, edit, and share documents simultaneously through web browsers. Documents are stored in distributed cloud storage with automatic synchronization and version control. The system ensures secure collaboration regardless of the user's location.

### Architecture Components: -

- **Clients:** Web browsers, desktop applications, and mobile devices.
- **Web APIs:** Synchronize document changes across devices.
- **Cloud Platform:** Document editing service and collaboration engine.
- **Distributed Cloud Storage:** Stores documents and maintains version history.
- **Network Infrastructure:** Global Content Delivery Network (CDN), load balancers, and cloud networking.
- **Security:** Encryption, identity management, authentication, and access permissions.



**Working Process: -**

1. Users log into the collaboration platform.
2. A document is opened using a web browser.
3. Every modification is sent to cloud services through APIs.
4. The collaboration engine synchronizes changes instantly.
5. Distributed storage saves each document version.
6. Multiple users can edit the same document simultaneously.
7. Access permissions determine who can view or edit documents.
8. Encryption protects data during storage and transmission.

**Cloud Services Used: -**

- **IaaS:** Storage servers, virtual machines, networking.
- **PaaS:** Collaboration engine, synchronization APIs, version control.
- **SaaS:** Online document editor.

**Advantages: -**

- Real-time collaboration.
- Automatic document synchronization.
- Version history and recovery.
- Global accessibility.
- High availability.
- Strong security and access control.

The Cloud-Based Document Collaboration System combines cloud storage, APIs, distributed computing, and security services to provide seamless document editing and sharing. It improves productivity and supports efficient teamwork across different locations.

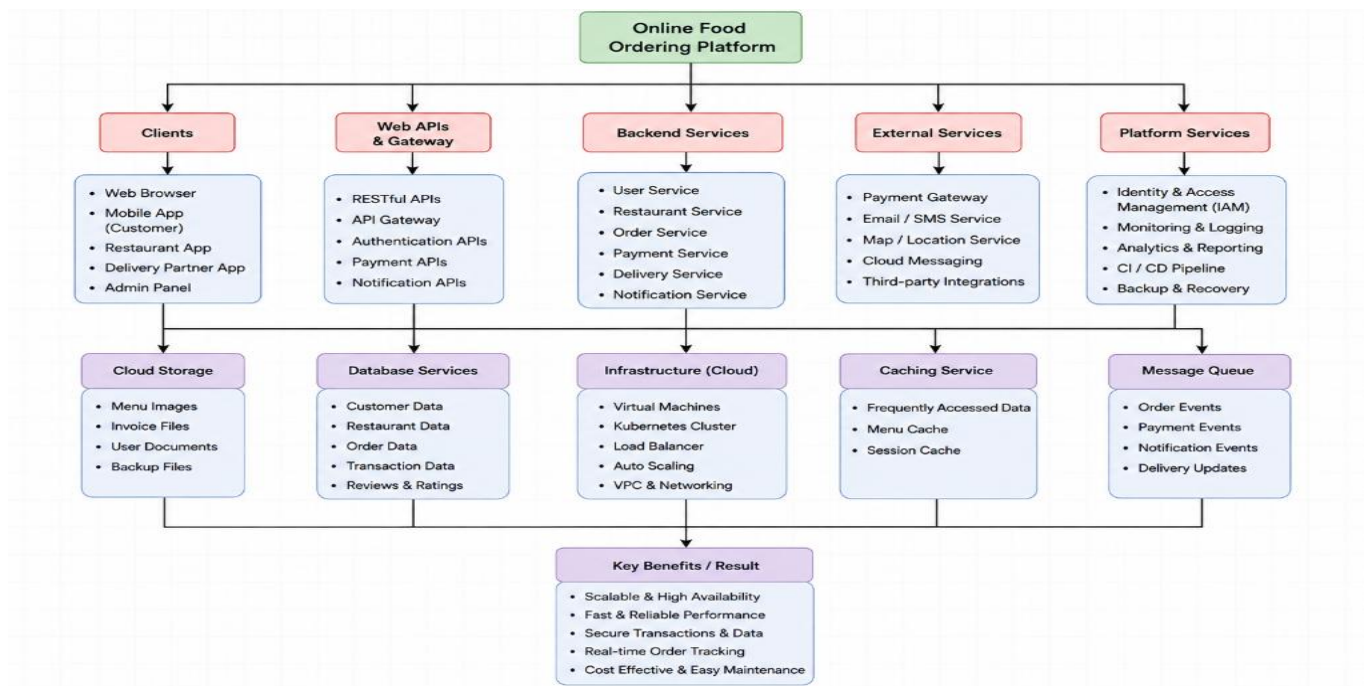
### 3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

An Online Food Ordering Platform is a cloud-based application that allows customers to browse restaurants, place orders, and make payments using web browsers or mobile applications. The system uses cloud services, REST APIs, Kubernetes, and cloud storage to provide scalable and secure food ordering services.

#### Architecture Components: -

- **Clients:** Web browsers and mobile applications.
- **Frontend:** User interface for customers, restaurants, and delivery partners.
- **REST APIs:** Communication between frontend and backend.
- **Backend Services:** Order management, payment, restaurant management, delivery tracking.
- **Cloud Storage:** Menu images, customer details, invoices, and order history.
- **Infrastructure:** Virtual machines, Kubernetes clusters, load balancers.
- **Security:** HTTPS, authentication tokens, and Role-Based Access Control (RBAC).



#### Working Process: -

1. Customers browse restaurants through the web or mobile app.
2. Menu details are retrieved using REST APIs.
3. Customers place food orders.
4. Backend services process the order.
5. Order details are securely stored in cloud databases.
6. Menu images and invoices are stored in cloud object storage.
7. Kubernetes automatically scales backend services during high traffic.
8. Authentication and RBAC secure customer and restaurant accounts.
9. Customers receive real-time order updates.

**Cloud Services Used: -**

- **IaaS:** Virtual machines, networking, storage.
- **PaaS:** Kubernetes, API management, database services.
- **SaaS:** Food ordering web application.

**Advantages: -**

- Supports thousands of concurrent users.
- Automatic scaling during peak hours.
- Secure online transactions.
- Fast response using cloud infrastructure.
- Reliable data storage and backup.
- Easy maintenance and deployment.

The Online Food Ordering Platform demonstrates effective integration of cloud infrastructure, Kubernetes, REST APIs, storage, and security services. It provides a scalable, reliable, and secure solution for customers, restaurants, and delivery partners while ensuring smooth real-time operations.